SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – APPLICATION BASED QUESTIONS

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Application Based Questions, In-class
Weightage:	_____ 40% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Tejhash P	**Reg. No.:**	192512495
Section / Batch:	1	**Date:**	11/10/2026

Code of Conduct

I Tejhash P, 192512495 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Tejhash.P

Instructions

- Read each question carefully before answering (in-Class Assessment)
- Each question is a real-world application scenario. Identify the AI technique that fits, apply it step by step, and justify every decision.
- Questions are drawn from Informed Search (Greedy, A*, heuristics), Local Search and Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Show intermediate tables (open lists, domains, α/β values) and state the final answer clearly.
- Marks will be awarded using the Application Based Questions rubric: understanding of problem context, application of concepts, problem-solving approach, accuracy of solution and clarity.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions (admissibility, consistency)	**Q1**
2. Local Search, Online Search & Constraint Satisfaction	Local search and optimization problems, online search agent, CSP (MRV, degree heuristic, forward checking)	**Q2, Q3**
3. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q4**

Rubrics for Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Problem Context	**20%**	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	**30%**	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	**20%**	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	**20%**	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	**10%**	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context (2)	Application of Concepts (3)	Problem-Solving Approach (2)	Accuracy of Solution (2)	Clarity (1)	Sub. Total (10)	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Application Based Questions

Question 1: Campus Delivery Robot – Greedy Best-First vs A*

Topic: Informed search strategies, heuristic functions | 15 minutes

A campus delivery robot must carry a parcel from the Store (S) to the Hostel (G). The one-way road network and the robot's heuristic estimate h(n) of the remaining travel cost to G (in minutes) are given below.

From	To	Cost
S	A	1
S	B	4
A	C	2
A	D	5
B	G	5
C	D	1
D	G	2

Node	S	A	B	C	D	G
h(n)	6	5	3	3	2	0

Assume ties are broken alphabetically and that a node is re-opened if a cheaper path to it is found.

(a) Apply Greedy Best-First Search from S to G. Write the order in which nodes are expanded, the path returned and its total cost.

(b) Apply A* search. Show the open list with g(n), h(n) and f(n) after each expansion, the order of expansion, and the final path and cost.

(c) Check whether h(n) is admissible and consistent by verifying the required condition for every node and every road.

(d) Compare the outputs of (a) and (b). Explain why Greedy Best-First Search returned a different result and which algorithm you would deploy for the robot, with justification.

Answer

(a) Greedy Best-First Search (f = h)

Step	Node expanded	Open list after expansion (node : h)
0	–	S:6
1	S	A:5, B:3
2	B (lowest h = 3)	A:5, G:0
3	G (h = 0): goal test passed	–

Order of expansion: **S, B, G**. Path returned: **S** → **B** → **G**. Total cost = 4 + 5 = **9** minutes.

(b) A* Search (f = g + h)

Step	Node expanded (f)	Open list after expansion, as node (g, h, f)
0	–	S(0, 6, 6)
1	S (6)	A(1, 5, 6), B(4, 3, 7)
2	A (6)	C(3, 3, 6), B(4, 3, 7), D(6, 2, 8)
3	C (6)	D(4, 2, 6) [cheaper via C, replaces D(6, 2, 8)], B(4, 3, 7)
4	D (6)	G(6, 0, 6), B(4, 3, 7)
5	G (6): goal test passed	–

Order of expansion: **S, A, C, D, G**. Final path: **S** → **A** → **C** → **D** → **G**, cost = 1 + 2 + 1 + 2 = **6** minutes. B (f = 7) is never expanded because every node on the optimal route has f = 6.

(c) Admissibility and consistency of h

Admissible: h(n) ≤ h*(n), the true cheapest cost from n to G.

Node	h(n)	True cost h*(n)	Cheapest route	h ≤ h*?
S	6	6	S-A-C-D-G	Yes
A	5	5	A-C-D-G (2+1+2)	Yes
B	3	5	B-G	Yes
C	3	3	C-D-G (1+2)	Yes
D	2	2	D-G	Yes
G	0	0	–	Yes

Consistent: for every road n → n′, h(n) ≤ c(n, n′) + h(n′).

Road	c(n, n′)	h(n)	c + h(n′)	h(n) ≤ c + h(n′)?
S → A	1	6	1 + 5 = 6	6 ≤ 6, Yes
S → B	4	6	4 + 3 = 7	6 ≤ 7, Yes
A → C	2	5	2 + 3 = 5	5 ≤ 5, Yes
A → D	5	5	5 + 2 = 7	5 ≤ 7, Yes
B → G	5	3	5 + 0 = 5	3 ≤ 5, Yes
C → D	1	3	1 + 2 = 3	3 ≤ 3, Yes
D → G	2	2	2 + 0 = 2	2 ≤ 2, Yes

Both conditions hold for every node and road, so h is **admissible and consistent**. Therefore A* is optimal and f never decreases along a path.

(d) Comparison and recommendation

	Greedy Best-First	A*
Order of expansion	S, B, G	S, A, C, D, G
Path	S → B → G	S → A → C → D → G
Cost	9 min	6 min (optimal)
Nodes expanded	3 (G is removed as goal)	5 (G is removed as goal)

Why they differ: Greedy looks only at h. After S it saw B with h = 3 against A with h = 5 and jumped to B, ignoring that the road S→B costs 4 and B→G costs 5, a total of 9. A* adds the cost already paid, g, so it saw that A (f = 6) is cheaper overall than B (f = 7) and found the longer-looking but cheaper route through A, C and D.

Deployment: use **A***. Its heuristic is admissible and consistent, so it guarantees the minimum delivery time (6 against 9 minutes) at the cost of only two more expansions. Greedy is faster but gives a 50% slower route.

Question 2: Wi-Fi Access Point Placement – Local Search and Online Search

Topic: Local search algorithms and optimization problems, online search agent | 15 minutes

A technician wants to place a Wi-Fi access point at the best position along a corridor with 10 candidate positions x = 0 to 9. The measured signal quality f(x) (higher is better) is:

x	0	1	2	3	4	5	6	7	8	9
f(x)	2	5	9	6	4	7	12	10	8	3

A move shifts the access point to an adjacent position (x − 1 or x + 1). Steepest-ascent hill climbing moves to the better neighbour and stops when no neighbour is strictly better.

(a) Run steepest-ascent hill climbing from the start positions x = 0, x = 4 and x = 9. Show each step and the final position and signal quality.

(b) Identify all local maxima and the global maximum. If the start position is chosen uniformly at random from the 10 positions, find the probability that one run reaches the global maximum, the probability that at least one of 3 random-restart runs does, and the expected number of runs needed.

(c) A simulated annealing search is at x = 2 and proposes the move to x = 3. Compute the acceptance probability e^(Δ/T) at T = 2 and at T = 1, and explain what this does to the chance of escaping the local maximum as T falls.

(d) The same search is later done by a robot that cannot see the whole corridor, can only measure f(x) where it stands, and cannot undo exploration for free. Explain why this is an online search problem, and state what the agent must remember to avoid cycling.

Answer

(a) Steepest-ascent hill climbing

Start	Step	Position x (f)	Neighbours (f)	Move
x = 0	1	0 (2)	1 (5)	to x = 1
	2	1 (5)	0 (2), 2 (9)	to x = 2 (best neighbour)
	3	2 (9)	1 (5), 3 (6)	none better: stop
x = 4	1	4 (4)	3 (6), 5 (7)	to x = 5 (best neighbour)
	2	5 (7)	4 (4), 6 (12)	to x = 6
	3	6 (12)	5 (7), 7 (10)	none better: stop
x = 9	1	9 (3)	8 (8)	to x = 8
	2	8 (8)	7 (10), 9 (3)	to x = 7
	3	7 (10)	6 (12), 8 (8)	to x = 6
	4	6 (12)	5 (7), 7 (10)	none better: stop

Results: from x = 0 it stops at **x = 2, f = 9** (a local maximum); from x = 4 it stops at **x = 6, f = 12**; from x = 9 it stops at **x = 6, f = 12**.

(b) Local and global maxima, and probabilities

Local maxima (better than both neighbours): **x = 2 (f = 9)** and **x = 6 (f = 12)**. The global maximum is **x = 6, f = 12**. Every other position has a better neighbour.

Ends at	Starting positions that lead there (basin)	Number
x = 2 (local, f = 9)	0, 1, 2, 3	4
x = 6 (global, f = 12)	4, 5, 6, 7, 8, 9	6

- One run reaches the global maximum: p = 6/10 = **0.6**.
- At least one of 3 random-restart runs: $1 - (1 - 0.6)^3 = 1 - 0.4^3 = 1 - 0.064 =$ **0.936**.
- Expected number of runs until the first success (geometric): 1/p = 1/0.6 = **1.67 runs**.

(c) Simulated annealing acceptance probability

The move x = 2 → x = 3 changes the value from 9 to 6, so Δ = 6 − 9 = −3 (a worse move). Acceptance probability = $e^{\Delta/T}$:

- At T = 2: $e^{-3/2} = e^{-1.5} \approx$ **0.223** (about 22%).
- At T = 1: $e^{-3} \approx$ **0.050** (about 5%).

Effect: as T falls, the chance of accepting a downhill move drops sharply, so the search becomes more like plain hill climbing and is increasingly trapped at x = 2. Escape is likely only while T is still high. To reach x = 6 it must also pass x = 4 (f = 4, a drop of 5 from 9), which has probability $e^{-5/2} \approx 0.082$ at T = 2 but only $e^{-5} \approx 0.007$ at T = 1. A slow cooling schedule keeps T high long enough to cross the valley.

(d) Online search version

This is an **online search** problem because the robot cannot see the whole corridor. It learns f(x) only by standing at a position and measuring, so it must **explore and act at the same time**; the result of a move is known only after making it, and exploring costs time and energy that cannot be undone for free. Offline planning on the whole table is impossible.

To avoid cycling the agent must remember: (1) the **positions already visited and the f value measured at each** (its map of the corridor); (2) **which neighbours of each position have been tried and which are unexplored**; (3) the **best position found so far** and a record of the previous positions, so it can backtrack and not step back and forth between the same two points. This is what an online DFS or LRTA*-style agent keeps (a result table and the visited states).

Question 3: Exam Timetable Scheduling – Constraint Satisfaction Problem

Topic: Constraint satisfaction problem (MRV, degree heuristic, forward checking) | 15 minutes

Five exams A, B, C, D and E must each be given one slot from {1, 2, 3}. Two exams that share students must not be in the same slot. The pairs sharing students are: A–B, A–C, A–D, B–C, B–D, C–E, D–E. Because of invigilator availability, exam A can only be in slots 1 or 2, and exam E only in slots 1 or 2.

Exam	Initial domain
A	{1, 2}
B	{1, 2, 3}
C	{1, 2, 3}
D	{1, 2, 3}
E	{1, 2}

Rules for the search: choose the variable using Minimum Remaining Values (MRV), break ties with the degree heuristic, then alphabetically; try values in ascending order; use forward checking after every assignment.

(a) Draw the constraint graph (list its edges) and compute the degree of every exam.

(b) Select the first exam to assign using MRV and the degree heuristic, justify the choice, assign its first value, and show the domains of all exams after forward checking.

(c) Continue the search with the same rules, showing the chosen exam, its value and the updated domains at each step, until a complete assignment is found. Verify it against every constraint.

(d) Interpret: explain how MRV, the degree heuristic and forward checking reduced the search effort here, and what forward checking would signal if a domain became empty.

Answer

(a) Constraint graph and degrees

Edges (7): **A–B, A–C, A–D, B–C, B–D, C–E, D–E**.

Exam	Neighbours	Degree
A	B, C, D	3
B	A, C, D	3
C	A, B, E	3
D	A, B, E	3
E	C, D	2

(b) First variable (MRV + degree heuristic)

Domain sizes: A = 2, B = 3, C = 3, D = 3, E = 2. MRV picks the smallest domains, which gives a tie between **A and E**. The degree heuristic breaks the tie by choosing the exam that constrains the most unassigned exams: A has degree 3 and E has degree 2, so **A is selected**. It is the most constrained exam and failure shows up early if it is going to happen. Assign its first value, **A = 1**.

Forward checking: A shares students with B, C and D, so slot 1 is removed from their domains.

Exam	A	B	C	D	E
Domain after A = 1	{1}	{2, 3}	{2, 3}	{2, 3}	{1, 2}

(c) Continuing the search

Step	Exam chosen (reason)	Value	Domains after forward checking
1	A (MRV tie with E, higher degree)	1	A{1}, B{2,3}, C{2,3}, D{2,3}, E{1,2}
2	B (MRV: B, C, D, E all size 2; remaining degrees all 2; alphabetical)	2	B conflicts with C and D, so 2 is removed from both: C{3}, D{3}, E{1,2}
3	C (MRV: size 1; tie with D; degrees tie; alphabetical)	3	Neighbours E: 3 is not in its domain, so E{1,2}; D{3} (C and D are not linked)
4	D (MRV: size 1)	3	E{1,2} unchanged (3 not present)

Step	Exam chosen (reason)	Value	Domains after forward checking
5	E (only exam left; first value)	1	Complete assignment

Complete assignment: A = 1, B = 2, C = 3, D = 3, E = 1, found with no backtracking.

Verification: A–B: 1 ≠ 2 □; A–C: 1 ≠ 3 □; A–D: 1 ≠ 3 □; B–C: 2 ≠ 3 □; B–D: 2 ≠ 3 □; C–E: 3 ≠ 1 □; D–E: 3 ≠ 1 □. Availability: A = 1 is in {1, 2} □ and E = 1 is in {1, 2} □. C and D may share slot 3 because they share no students.

(d) Interpretation

- **MRV** chose the most constrained exam first (A, only two slots), so the search fails early rather than late if a conflict exists.
- **Degree heuristic** broke the A/E tie in favour of the exam involved in more constraints (A, degree 3), which removes the most options from the others.
- **Forward checking** immediately removed conflicting slots, which reduced C and D to single values, so MRV picked them next and no wrong guesses were made. Result: 5 assignments, 0 backtracks.

Empty domain: if forward checking ever left an unassigned exam with no legal slot, it signals a dead end: the current assignment cannot be completed, so the search must backtrack at once, undo the last assignment and try its next value, without exploring further below it.

Question 4: Board-Game AI – Minimax, Alpha-Beta Pruning and Evaluation Function

Topic: Game playing, adversarial search, evaluation functions | 15 minutes

A game-playing agent (MAX) searches two plies ahead: MAX chooses one of three moves (B, C, D), then MIN replies with one of three moves. Positions at depth 2 are scored by the evaluation function

$$\mathbf{Eval = 2 \times M + 1 \times Mo}$$

where M is the material advantage of MAX and Mo is the mobility advantage of MAX. The leaf features (M, Mo) are given for the leaves in left-to-right order:

MIN node	Leaf 1 (M, Mo)	Leaf 2 (M, Mo)	Leaf 3 (M, Mo)
B	(1, 1)	(5, 2)	(3, 2)
C	(1, 0)	(1, 2)	(2, 2)
D	(6, 2)	(2, 1)	(0, 2)

(a) Compute the Eval value of all nine leaves.

(b) Run plain minimax. Give the backed-up value of B, C and D, the value at the root and the move MAX should play.

(c) Run alpha-beta pruning on the same tree (left to right). Show α and β at each node, mark every pruned branch, and state the root value and the number of leaves evaluated.

(d) If the leaves under C were ordered (6, 4, 2) instead, how many leaves would alpha-beta evaluate? Explain the effect of move ordering and why pruning never changes the minimax value.

Answer

(a) Leaf values, Eval = 2 × M + Mo

MIN node	Leaf 1	Leaf 2	Leaf 3
B	(1, 1): 2×1 + 1 = **3**	(5, 2): 2×5 + 2 = **12**	(3, 2): 2×3 + 2 = **8**
C	(1, 0): 2×1 + 0 = **2**	(1, 2): 2×1 + 2 = **4**	(2, 2): 2×2 + 2 = **6**
D	(6, 2): 2×6 + 2 = **14**	(2, 1): 2×2 + 1 = **5**	(0, 2): 2×0 + 2 = **2**

(b) Plain minimax

- MIN node B = min(3, 12, 8) = **3**
- MIN node C = min(2, 4, 6) = **2**
- MIN node D = min(14, 5, 2) = **2**
- Root (MAX) = max(3, 2, 2) = **3**, so MAX should play **move B**. All 9 leaves are evaluated.

(c) Alpha-beta pruning (left to right)

Node	α, β on entry	Leaf values seen	Updates / action	Returns
Root (MAX)	−∞, +∞	–	starts with B	–
B (MIN)	−∞, +∞	3, 12, 8	after 3: β = 3; 12 and 8 do not lower it; no cut-off	3
Root	α = 3	–	α updated to 3	–
C (MIN)	3, +∞	2 (leaf 1)	2 ≤ α = 3, so C can be at most 2: **cut-off, prune leaves 2 and 3** (values 4, 6)	2
D (MIN)	3, +∞	14, 5, 2	after 14: β = 14; after 5: β = 5 (> α, continue); leaf 3 = 2 ≤ α gives 2 (last leaf, nothing left to prune)	2
Root	α = 3	–	max(3, 2, 2) = 3	**3**

Root value = 3 (move B, the same as minimax). **Pruned branches:** leaf 2 and leaf 3 under C. **Leaves evaluated: 3 + 1 + 3 = 7 of 9.**

(d) Effect of move ordering

Reading (6, 4, 2) as the Eval values of C's leaves in that order: C starts with α = 3. Leaf 1 = 6 gives β = 6, still above α. Leaf 2 = 4 gives β = 4, still above α. Leaf 3 = 2 finally falls below α. No cut-off happens before the last leaf, so all 3 leaves under C are evaluated and the total becomes 3 + 3 + 3 = **9 leaves (no pruning)**.

Move ordering: pruning at a MIN node happens only when a low value is found early. With C ordered (2, 4, 6) one leaf was enough; with (6, 4, 2) the low value comes last and nothing is saved. With the best ordering at every MIN node (for example D as 2, 5, 14 as

well) only 3 + 1 + 1 = 5 leaves would be needed, so good ordering (best moves first) can nearly halve the search depth cost.

Why the value never changes: a branch is cut only when its value is already provably unable to affect the parent: at C the value is at most 2, which is below α = 3 that MAX can already get from B, so MAX will never choose C whatever the other leaves are. Pruned leaves therefore cannot change the backed-up minimax value (3) or the chosen move; alpha-beta only saves work.